



GRAPH MACHINE LEARNING FOR ARCHITECTURAL DATA

Professor Wassim Jabi, PhD

GRAPH MACHINE LEARNING FOR ARCHITECTURAL DATA

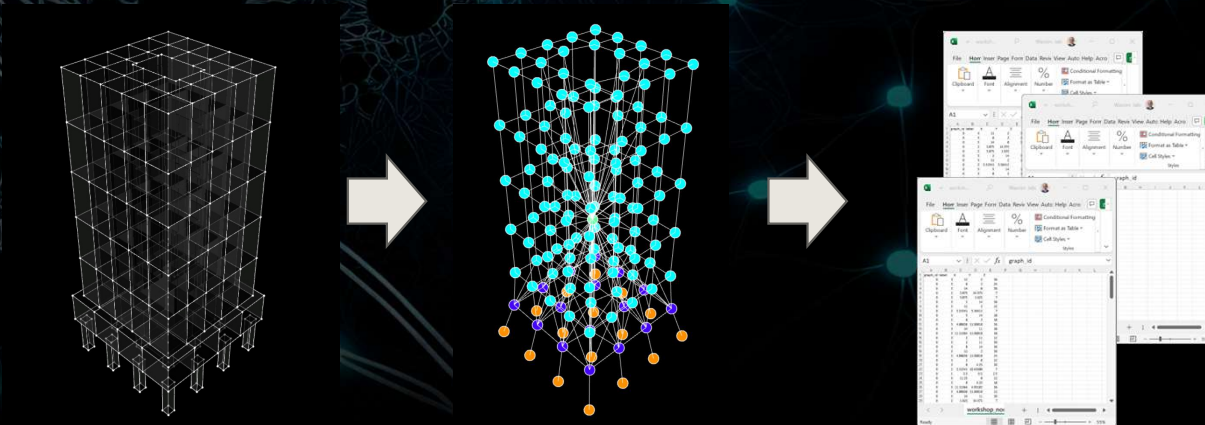
- Graph Machine Learning enables the analysis of relational structures in buildings.
- It transforms spatial models into graph-based representations for predictive tasks.

MOTIVATION

- Buildings are relational systems where connections matter as much as objects.
- Problems such as circulation depend on relationships.
- Traditional ML fails to capture these dependencies.

FROM GEOMETRY TO GRAPHS

- Rooms and elements can be represented as nodes.
- Adjacency and visibility are represented as edges.
- Attributes enrich nodes, edges, and graphs.



WHAT IS GRAPH MACHINE LEARNING

- GML operates on structured relational data.
- It learns from both features and topology.
- It is suited to complex systems like buildings.

GML VS CLASSICAL ML

- Classical ML assumes independent samples.
- Graph ML models interdependencies.
- Graph ML handles non-Euclidean structures.

WHY GML IN ARCHITECTURE

- Architecture is inherently relational.
- Graph ML captures connectivity and hierarchy.
- It aligns with spatial reasoning.

WHEN TO USE GML

- Use when relationships influence outcomes.
- Ideal for spatial and networked data.
- Avoid for purely tabular datasets.

TYPES OF GML TASKS

- Graph-level predicts whole-graph properties.
- Node-level predicts per-node attributes.
- Edge-level predicts relationship properties.
- Link prediction infers missing edges.

TERMINOLOGY

- **Graph:** The representation of a single entity/datpoint (a school, a library etc).
- **Dataset:** The set of graphs (e.g. for training, validation, testing, or prediction).
- **Model:** The graph neural network being trained or being used to predict.
- **Accuracy:** Computes the accuracy of the input predictions based on the input labels. This is to be used only with classification not with regression.
- **RMSE:** Computes the accuracy based on the mean squared error of the input predictions based on the input actual values. This is to be used only with regression not with classification..
- **Unbalanced dataset:** Where the numbers of entities in each category in a dataset are not equal.
- **Graph vs. PyG Graph:** Topologic Graph vs. a Pytorch Geometric graph.
- **Hyperparameters:** The parameters for the ML model that affect its accuracy and performance.
- **Confusion Matrix:** A technique for summarizing the performance of a classification algorithm.

TASKS

- **Graph Classification:** Is this graph a hospital or a library or a school? What category does it fall into?
- **Graph Regression:** What is the annual energy consumption of this building? What value does this building have on a continuous line of possible values.
- **Node Classification:** Is this room a bedroom or a toilet? Predict the label of a node.
- **Node Regression:** What is the predicted cooling load of this room given the cooling loads of other rooms? Predict the value of node property based on other node properties on a continuous line of possible values.
- **Edge Classification:** What type of wall connects the bedroom and the bathroom? Predict the label of the edge between two nodes.
- **Edge Regression:** Predict the U-Value of the wall connecting the courtyard to the lobby? Predict the value of the edge along a continuous line of possible values.
- **Edge Prediction:** Is the bedroom adjacent to the bathroom? Predict if an edge exists or not between two nodes.

HYPERPARAMETERS

Optimizer

A gradient-based optimisation algorithm used to update model weights during training.

Supported options: "adam" or "adamw" (adaptive methods with optional weight decay).

Learning Rate (lr)

A scalar parameter that controls the step size taken during each optimisation update.

Smaller values lead to more stable but slower convergence, while larger values may cause instability.

Task

Defines the type of prediction problem.

Supported options:

"classification" → predicts discrete labels

"regression" → predicts continuous values

"link_prediction" → predicts existence of edges

Level

Defines what the model predicts on.

Supported options:

"graph" → one prediction per graph

"node" → one prediction per node

"edge" → one prediction per edge

"link" → predicts missing edges

Cross-Validation Method (cv)

Defines how the dataset is split.

Supported options:

"holdout" → single train/validation/test split

"kfold" → k-fold cross-validation (**graph-level only**)

Hidden Layer Widths (hidden_dims)

A tuple defining the size of each hidden layer in the GNN encoder.

Example: (16, 32, 16) creates three layers with 16, 32, and 16 features respectively.

Convolution Layer (conv)

Defines the graph convolution operator used in the encoder.

Supported options:

"sage" → GraphSAGE (sampling-based, scalable)

"gcn" → Graph Convolutional Network

"gatv2" → Graph Attention Network (v2)

A graph convolution aggregates information from neighbouring nodes to update node representations.

Activation Function (activation)

Defines the non-linear transformation applied after each layer.

Supported options: "relu", "gelu", "elu"

Pooling (pooling) (graph-level only)

Defines how node embeddings are aggregated into a graph-level representation.

Supported options:

"mean" → average pooling

"max" → max pooling

"sum" → additive pooling

Pooling reduces node-level information into a single graph embedding.

Batch Size (batch_size)

The number of graphs processed per training step (graph-level tasks).

For node/edge tasks, batching is effectively one graph at a time.

HYPERPARAMETERS

Epochs (epochs)

The number of full passes through the training dataset during optimisation.

Weight Decay (weight_decay)

A regularisation parameter that penalises large weights to reduce overfitting.

Dropout (dropout)

A regularisation technique that randomly deactivates neurons during training to improve generalisation.

Batch Normalisation (batch_norm)

A boolean flag indicating whether to normalise layer outputs to stabilise training.

Residual Connections (residual)

A boolean flag enabling skip connections between layers to improve gradient flow in deeper networks.

Gradient Clipping (gradient_clip_norm)

Limits the magnitude of gradients during backpropagation to prevent instability.

Early Stopping (early_stopping)

Stops training when validation performance no longer improves.

Early Stopping Patience (early_stopping_patience)

Number of epochs to wait before stopping after no improvement.

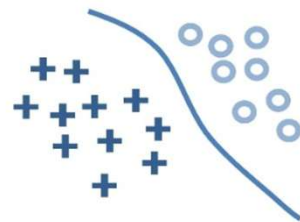
Loss Function

Determined automatically based on the task:

- **Classification** → Cross-Entropy Loss
- **Regression** → Mean Squared Error (MSE)
- **Link Prediction** → Binary Cross-Entropy with logits

CLASSIFICATION VS REGRESSION

- Classification predicts categories.
- Regression predicts continuous values.
- Choice depends on output type.



Classification



Regression

DATA REPRESENTATION IN PYG

- Node features are matrices.
- Edges define connectivity.
- Labels define prediction targets.

MODEL ARCHITECTURE

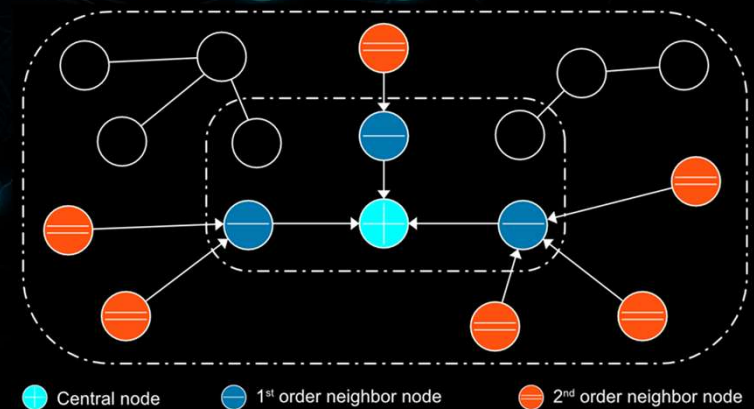
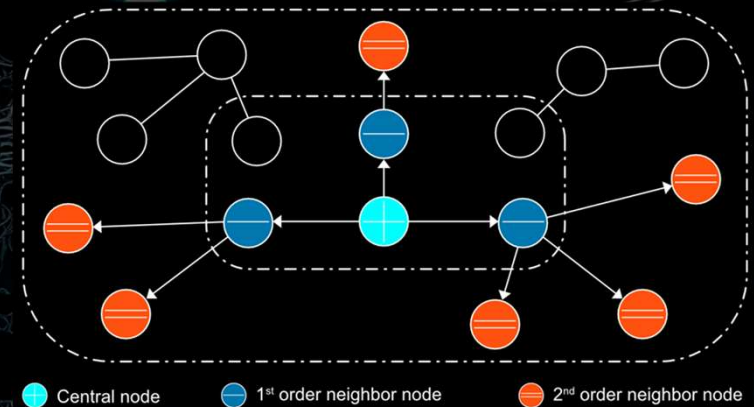
- Encoder learns embeddings.
- Head maps embeddings to outputs.
- Architecture is modular.

GRAPH CONVOLUTIONS

- Aggregate neighbour information.
- Update node representations.
- Capture local structure.

SAGE CONVOLUTION

- Samples neighbourhoods.
- Scales to large graphs.
- Supports inductive learning.



FEATURE ENGINEERING

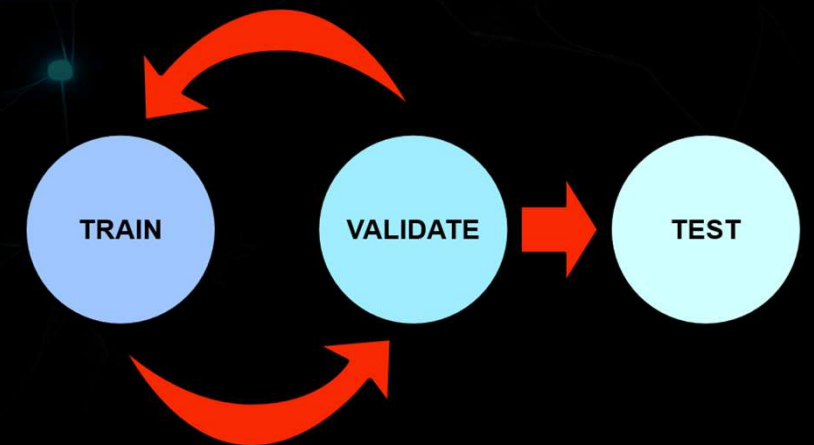
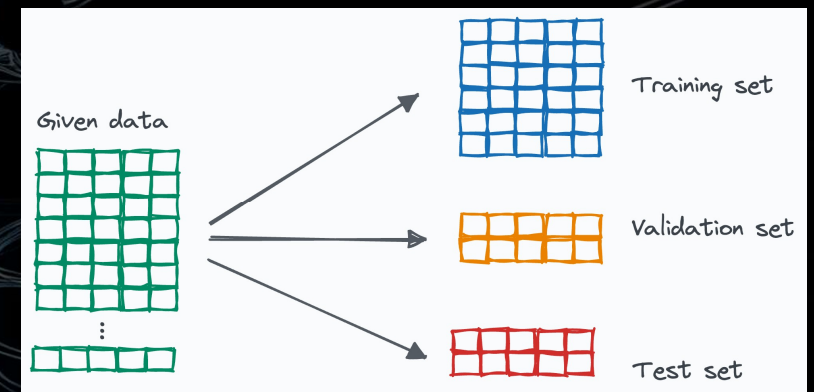
- Node features encode geometry and semantics.
- Edge features encode relationships.
- Feature design affects performance.

TRAINING PIPELINE

- Load and structure data.
- Split dataset.
- Train model iteratively.

TRAIN, VALIDATION, TEST

- Training updates parameters.
- Validation tunes model.
- Test evaluates final performance.

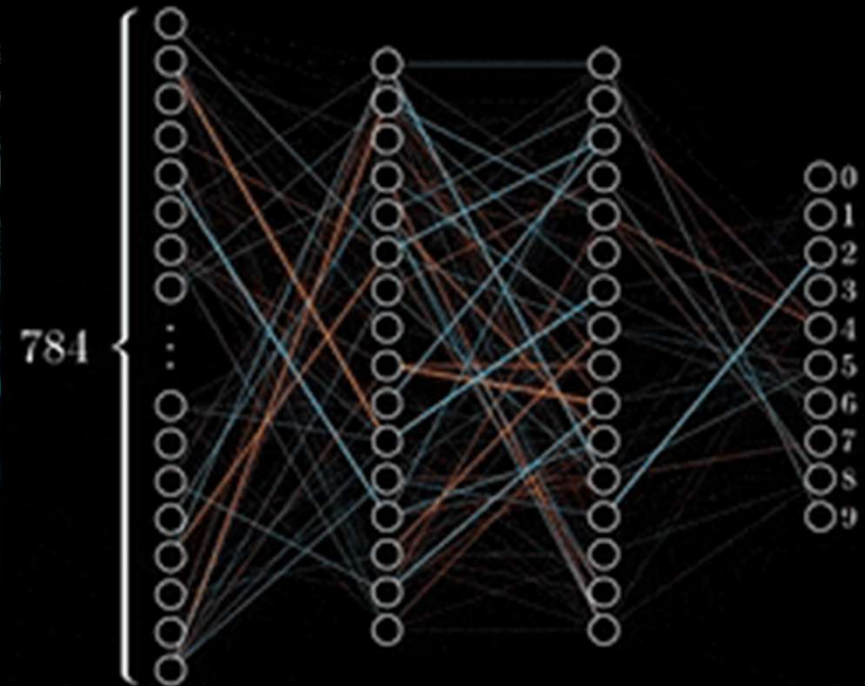


SPLITTING STRATEGIES

- Graph-level splits graphs.
- Node-level uses masks.
- Link prediction splits edges.

TRAINING LOOP

- Forward pass computes predictions.
- Loss measures error.
- Backpropagation updates weights.



VALIDATION DURING TRAINING

- Validation checks generalisation.
- Detects overfitting.
- Enables early stopping.

HYPERPARAMETERS

- Define model behaviour.
- Include learning rate and layers.
- Require careful tuning.

ARCHITECTURAL HYPERPARAMETERS

- Conv type controls aggregation.
- Layer size controls capacity.
- Dropout stabilises training.

HYPERPARAMETER TUNING

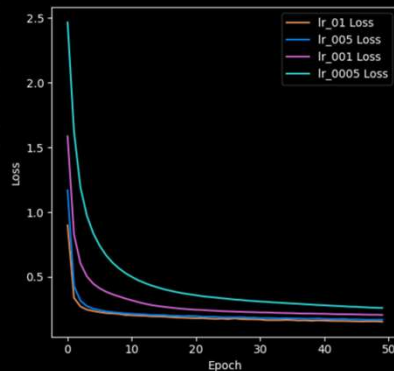
- Grid search is exhaustive.
- Random search is efficient.
- Trade-off accuracy vs cost.

ADVANCED TUNING

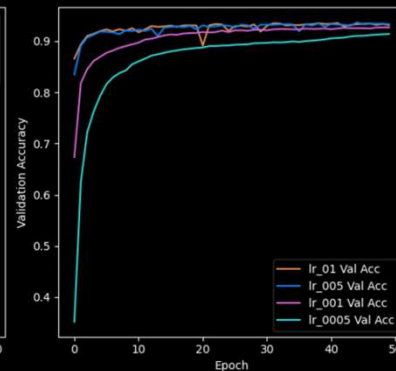
- Bayesian optimisation guides search.
- Hyperband allocates resources efficiently.
- Improves tuning efficiency.

LOSS FUNCTIONS

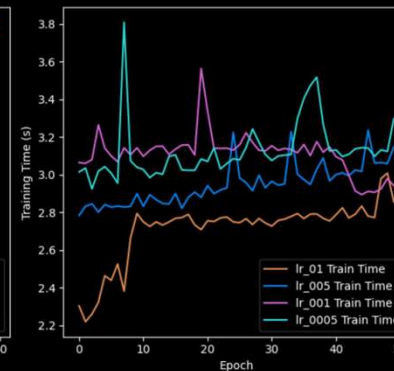
- Cross-entropy for classification.
- MSE for regression.
- Measures prediction error.



a) Loss Comparison



b) Validation Accuracy Comparison



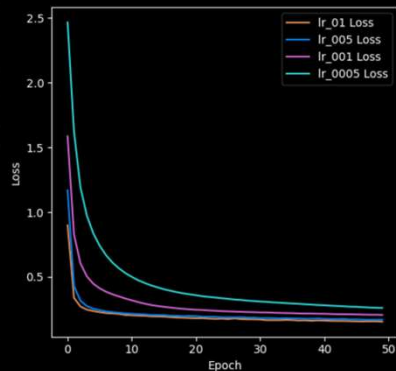
c) Runtime Comparison

	Accuracy	Precision	Recall	F1 Score
Units = 8	92.56	73.90	71.82	70.86
Units = 16	92.53	77.90	75.16	75.17
Units = 32	92.57	80.81	74.74	74.69
Units = 64	92.67	84.16	77.62	78.32

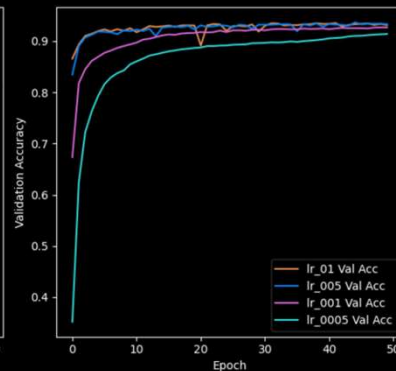
	Accuracy	Precision	Recall	F1 Score
lr = 0.01	92.66	85.19	81.95	82.98
lr = 0.005	92.71	84.25	77.07	77.49
lr = 0.001	92.64	88.07	83.16	84.80
lr = 0.0005	92.72	85.41	79.36	80.57

METRICS (CLASSIFICATION)

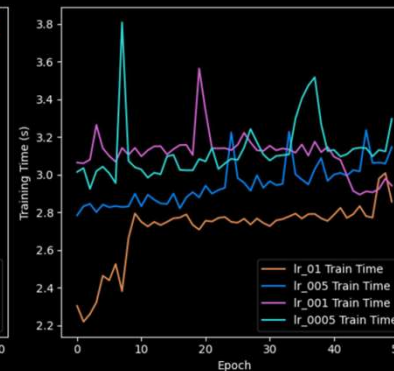
- Accuracy measures correctness.
- Precision measures false positives.
- Recall measures false negatives.
- F1 balances both.



a) Loss Comparison



b) Validation Accuracy Comparison



c) Runtime Comparison

	Accuracy	Precision	Recall	F1 Score
Units = 8	92.56	73.90	71.82	70.86
Units = 16	92.53	77.90	75.16	75.17
Units = 32	92.57	80.81	74.74	74.69
Units = 64	92.67	84.16	77.62	78.32

	Accuracy	Precision	Recall	F1 Score
lr = 0.01	92.66	85.19	81.95	82.98
lr = 0.005	92.71	84.25	77.07	77.49
lr = 0.001	92.64	88.07	83.16	84.80
lr = 0.0005	92.72	85.41	79.36	80.57

METRICS (REGRESSION)

- MAE measures average error.
- RMSE penalises large errors.
- R^2 measures explained variance.

OVERFITTING AND BEST PRACTICE

- Overfitting harms generalisation.
- Regularisation mitigates it.
- Proper validation is essential.

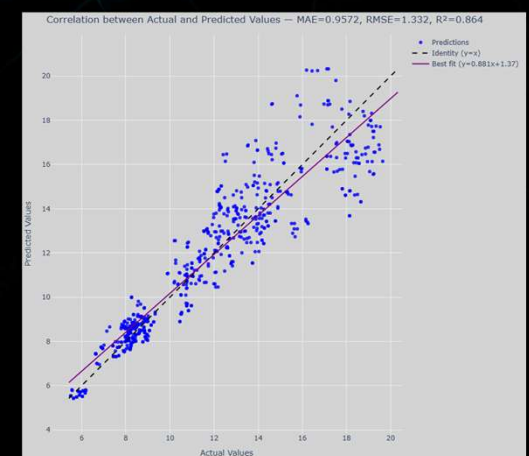
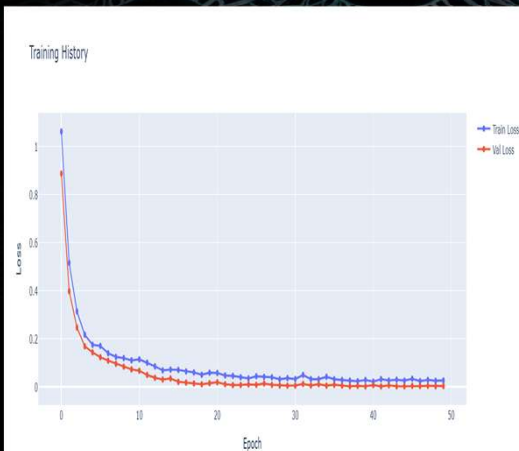
CONFUSION MATRIX

- Shows predicted vs true classes.
- Reveals error patterns.
- Useful for imbalance.

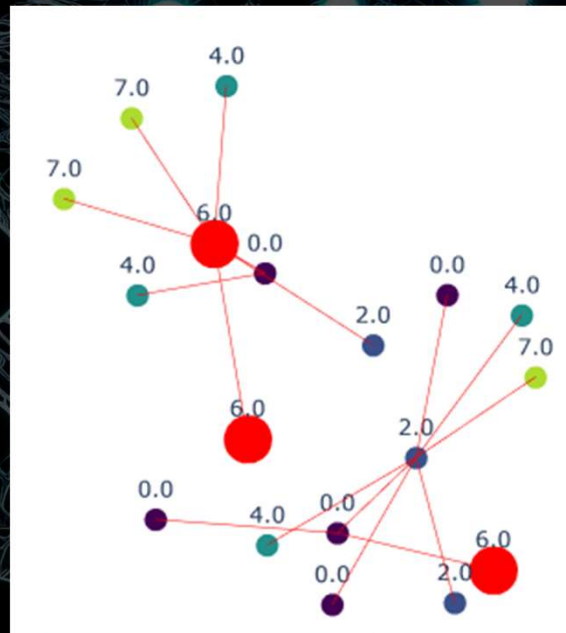


VISUALISATION

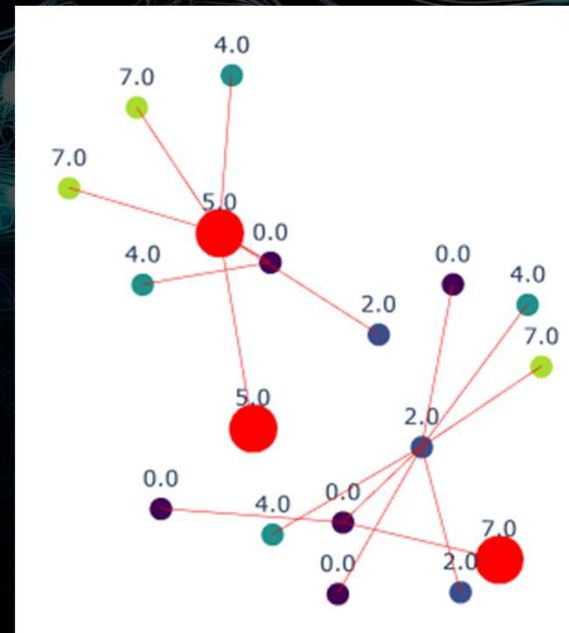
- Loss curves track training.
- Confusion matrices show errors.
- Scatter plots compare predictions.



NODE CLASSIFICATION VISUALISATION



TRUE



PREDICTION

THANK YOU

- Wassim Jabi
- wassim.jabi@gmail.com
- topologic.app